

Tabular Q-Learning Control of the Lorenz System

Implementation, Design Decisions, Results, and Interpretation

Project documentation

September 3, 2026

Abstract

This project applies tabular Q-learning to a continuously evolving Lorenz system by discretizing both state and control. The final implementation is a self-contained NumPy program: it simulates the controlled dynamics, constructs a reward that exactly matches a trajectory-level reference objective, trains an ϵ -greedy Q-table, and evaluates the frozen greedy policy. For seed 7, the learned unregularized policy reduces task loss by 45.3% relative to no control. Adding a control-effort penalty reduces effort by 31.7% relative to the unregularized policy, with a smaller task-loss improvement. This report concentrates on those design choices, the code path, the experiment, and correct interpretation of the plots; only the minimum Lorenz and Q-learning background is included.

1 Project goal and system definition

The goal is to learn a feedback policy that changes the Lorenz trajectory so that x spends more of the finite horizon in the positive region, optionally while limiting control effort. The controlled dynamics are

$$\dot{x} = 10(y - x) + u, \quad \dot{y} = x(28 - z) - y, \quad \dot{z} = xy - \frac{8}{3}z. \quad (1)$$

Only the x equation is actuated, but the full state (x, y, z) is observed. Forward Euler integration with $\Delta t = 0.01$ gives the transition used by the learning environment. The fixed initial state is $(0, 1, 1.05)$, and one episode lasts one Lyapunov time under the project's convention:

$$N = \text{round}\left(\frac{1}{0.9056 \Delta t}\right) = 110$$

transitions, producing 111 recorded states. This is a short controlled transient, not a reconstruction of the long-run Lorenz attractor.

2 Most important design decisions

2.1 Continuous physics, discrete Q-table

Tabular Q-learning requires finite states and actions. The physical simulator remains continuous, while the agent sees a uniform $15 \times 15 \times 15$ grid over

$$x, y \in [-30, 30], \quad z \in [0, 60].$$

Each bin is four units wide. Values outside these ranges are clipped to an edge bin; therefore distinct out-of-range states become indistinguishable. Nine uniformly spaced controls are available:

$$\mathcal{A} = \{-60, -45, -30, -15, 0, 15, 30, 45, 60\}.$$

The Q-table shape is $15 \times 15 \times 15 \times 9$, or 30,375 `float64` values (243,000 bytes). This is small and transparent, but every state inside a four-unit cube shares the same action values and the table cannot interpolate to unseen bins.

2.2 Reward is aligned with the evaluation objective

The state cost is a smooth penalty for negative x ,

$$c(x) = \frac{1}{2} [1 - \tanh(x/2)], \quad J_x = \frac{1}{N+1} \sum_{k=0}^N c(x_k).$$

The optional normalized control effort is

$$J_u = \frac{1}{N} \sum_{k=0}^{N-1} \left(\frac{u_k}{60} \right)^2, \quad J = J_x + 0.007 J_u.$$

For transition k , the implementation returns

$$r_k = - \left[\frac{c(x_{k+1}) + \mathbf{1}_{k=0} c(x_0)}{N+1} + \mathbf{1}_{\text{reg}} \frac{0.007}{N} \left(\frac{u_k}{60} \right)^2 \right].$$

The initial-state cost is inserted into the first reward. Consequently, over a complete episode, $\sum_k r_k = -J$ exactly apart from floating-point roundoff. This is why the discount factor is $\gamma = 1$: discounting would change the weighting over time and break equivalence to the reference trajectory mean.

This alignment is the central scientific design choice. A high return now means a low reported objective, so training and evaluation optimize the same quantity. The identity does not generally hold if `max_steps` truncates the episode or divergence ends it early because the reward denominators still use the configured full horizon.

2.3 Exploration and deterministic evaluation

Training uses ϵ -greedy action selection. The schedule begins at $\epsilon = 1$, multiplies it by 0.99 after every episode, and floors it at 0.05 (reached after roughly 299 episodes). Thus training returns remain noisy even late in training. Evaluation sets exploration aside, chooses `argmax` actions, and does not update the table or decay ϵ .

The Q-table is initialized to zero. NumPy's `argmax` selects the first maximum, so an unseen state bin selects action index 0, corresponding to $u = -60$. This deterministic tie bias is simple but potentially strong; random tie-breaking or optimistic initialization would be reasonable alternatives.

3 How the code is organized

The final artifact is a single 617-line NumPy program, `sim.py`. It is tabular Q-learning, not a neural-network or PyTorch implementation.

Table 1: Implementation map.

Component	Responsibility
<code>phi</code> , <code>calculate_loss</code> <code>control_effort</code> , <code>reference_objective</code> <code>LorenzEnvEuler</code>	Smooth state indicator and trajectory task loss J_x . Independent evaluation of J_u and J .
<code>StateDiscretizer</code> <code>QLearningAgent</code>	State ownership, action mapping, Euler transition, reward, horizon, and divergence termination. Validation, clipping, uniform binning, and optional flat state index.
<code>train_q_learning</code> <code>evaluate_q_learning</code>	Q-table, local seeded random generator, ϵ -greedy selection, TD update, and exploration decay. Episode interaction loop and training-history collection.
<code>main</code>	Frozen greedy rollouts, trajectories, controls, losses, effort, objective, return, length, and divergence status. Command-line arguments, default construction, training, evaluation, and summary.

3.1 Environment path

reset(x_0) copies and validates a three-element initial state, resets the step counter, and caches $c(x_0)$. **step**(**action**) validates the action index, maps it to the physical control, executes one Euler step, builds the aligned reward, and returns (**next_state**, **reward**, **done**, **info**). Termination occurs at 110 steps or when a state contains NaN or exceeds the configured norm threshold. The final default threshold is infinite, so only NaN and the horizon are active in ordinary runs. Returned state arrays are copies, preventing callers from mutating environment state accidentally.

3.2 Discretizer and Q-table path

For state coordinate s_i , lower bound ℓ_i , width w_i , and bin count n_i , the implementation computes

$$b_i = \min\left(n_i - 1, \left\lfloor \frac{\text{clip}(s_i, \ell_i, h_i) - \ell_i}{w_i} \right\rfloor\right).$$

The upper clamp handles a state exactly on the upper boundary. The initial state maps to bin (7, 7, 0). The bin tuple indexes the first three Q-table dimensions; the action index supplies the fourth.

The implemented Q-learning update is

$$Q(s, a) \leftarrow Q(s, a) + 0.15 \left[r + \gamma(1 - d) \max_{a'} Q(s', a') - Q(s, a) \right],$$

where d is true on a terminal transition and $\gamma = 1$. The terminal case does not bootstrap from the following state. Although the method is off-policy, it learns from the greedy target while behavior remains ϵ -greedy.

3.3 Training and evaluation data flow

Listing 1: Essential training loop, corresponding to the implementation.

```
for each episode:
    state = env.reset()
    while not done and below optional max_steps:
        action = agent.select_action(state, training=True)
        next_state, reward, done, info = env.step(action)
        agent.update(state, action, reward, next_state, done)
        state = next_state
        accumulate return, length, and divergence
    agent.decay_epsilon()
record a trailing 100-episode mean return
```

Training returns equal-length arrays for episode return, length, post-decay ϵ , divergence, and rolling mean. Evaluation follows the same interaction path without **update**; it stores every state, action index, physical control, and the independently recomputed metrics. A trajectory has one more row than its action/control array because the initial state is included.

Constructor and helper validation rejects malformed state shapes, invalid bounds or bin counts, inconsistent hyperparameter aliases, actions outside the discrete range, continuous environments paired with the tabular agent, mismatched action counts, and nonpositive episode or step counts. **render** and **close** are placeholders; **TIMESTEPS=5000** is declared but unused.

4 Experimental protocol

Two separate agents were trained with the same seed, one without and one with control regularization. A no-control baseline used the same state, horizon, and integration, with $u = 0$. The five configured evaluation episodes are identical because the initial state, environment, and greedy policy are deterministic; they are repeated rollouts, not five independent samples.

Table 2: Final experiment configuration.

Group	Parameter	Value
Dynamics	$(\rho, \sigma, \beta); \Delta t$	(28, 10, 8/3); 0.01
Episode	initial state; horizon	(0, 1, 1.05); 110 transitions
Objective	$\varepsilon; U_{\text{ref}}; \lambda$	2; 60; 0.007
State grid	bounds; bins	$[-30, 30]^2 \times [0, 60]; 15^3$
Action grid	count; range	9; $[-60, 60]$
Learning	learning rate; discount	0.15; 1.0
Exploration	start; decay; floor	1.0; 0.99; 0.05
Run	train episodes; eval episodes; seed	600; 5; 7

5 Results

Table 3: Reproduced seed-7 results. Lower loss, effort, and objective are better.

Metric	No control	Unregularized	Regularized
Task loss J_x	0.537609	0.293970	0.344942
Normalized effort J_u	0.000000	0.726705	0.496023
Optimized objective J	0.537609	0.293970	0.348415
Evaluation return	–	-0.293970	-0.348415
Final-50 mean training return	–	-0.426024	-0.387902
Touched state bins	–	330 (9.78%)	289 (8.56%)
Nonzero Q entries	–	2,213 (7.29%)	1,884 (6.20%)
Diverged	no	no	no

The unregularized policy reduces task loss by 45.3% relative to no control. The regularized policy reduces it by 35.8%. Relative to the unregularized policy, regularization reduces normalized effort by 31.7% but raises task loss by 17.3%. This is the intended tradeoff. The reward check gives $|G + J| = 5.55 \times 10^{-17}$ for both learned policies, confirming the full-horizon reward/objective identity.

The result is evidence that the learned table controls this specific finite-horizon, fixed-initial-state problem. It is not evidence of global stabilization or a policy that generalizes across the attractor. Fewer than 10% of state bins were touched, which is the strongest numerical warning about generalization.

6 How to interpret the graphs

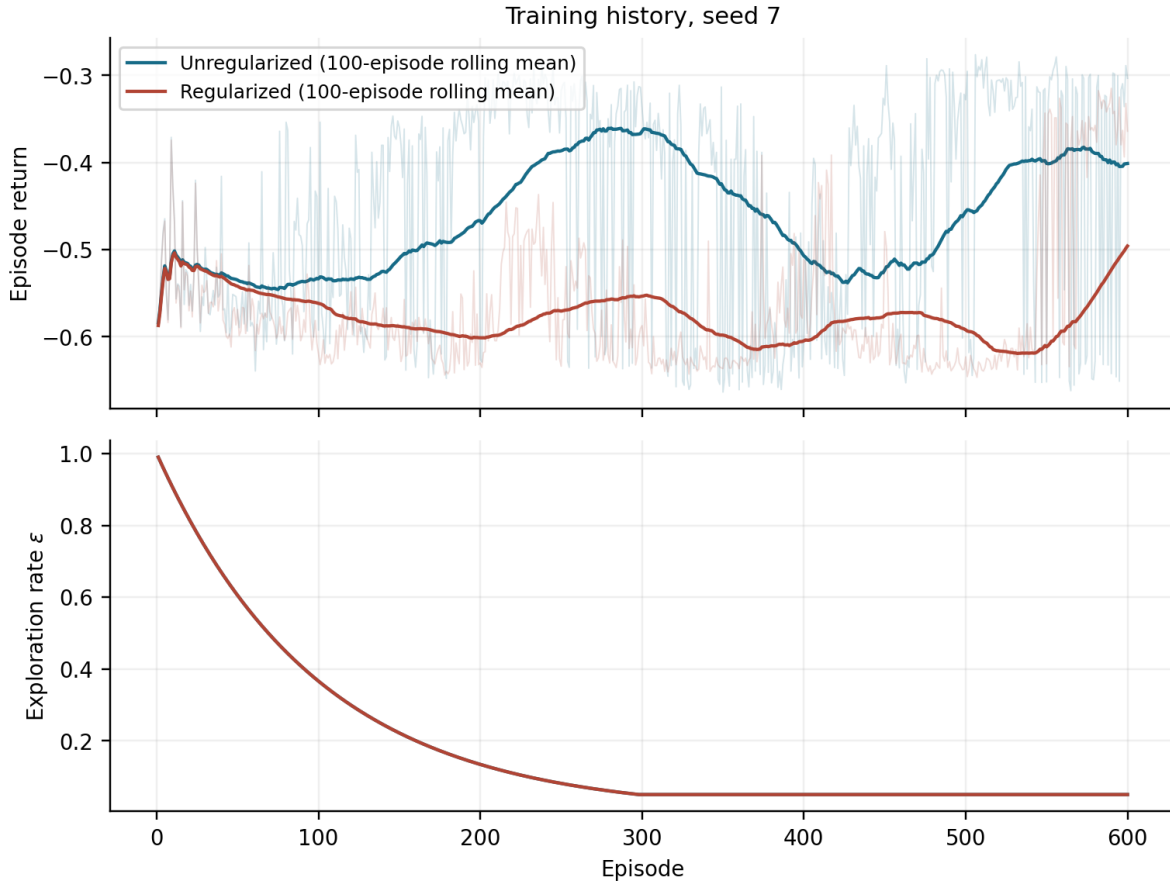


Figure 1: Raw episode returns (faint), trailing 100-episode means (solid), and the shared exploration schedule.

Training history (Figure 1). Less-negative return is better. The solid curve is the useful trend; the faint curve includes exploratory actions and therefore need not improve monotonically. Residual variation after episode 300 is expected because ϵ remains at 0.05 and Lorenz dynamics amplify small changes. The regularized and unregularized returns are not directly identical metrics: only the regularized curve includes effort cost. Final greedy results in Table 3 should be used to compare policies.

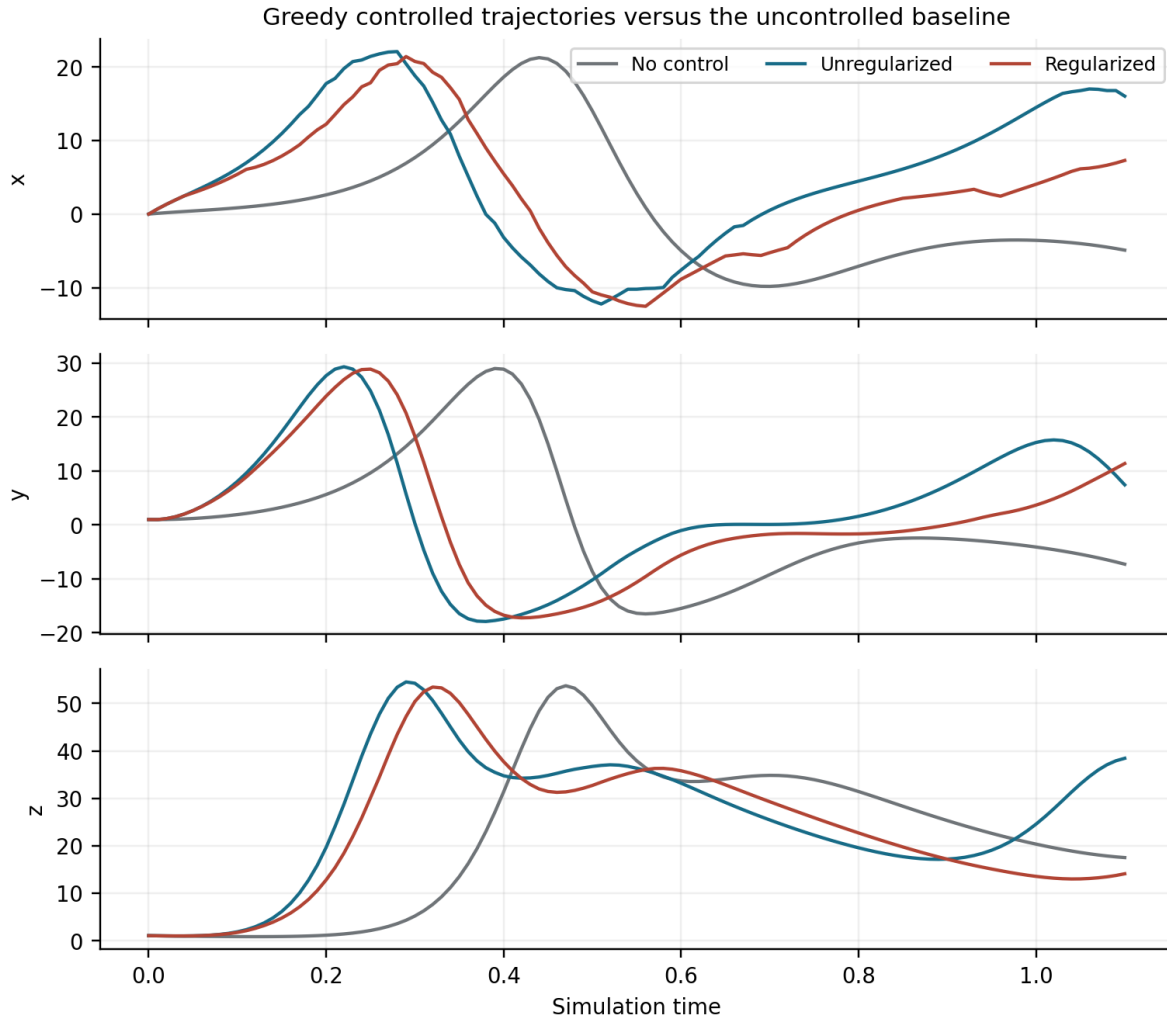


Figure 2: Uncontrolled and greedy controlled state trajectories over one episode.

State trajectories (Figure 2). The loss depends directly only on x : positive x contributes little cost and negative x contributes high cost. The controller changes x through u , while y and z respond through the coupled dynamics and also determine future actions through the state bin. A controlled curve need not resemble the baseline; separation is an expected consequence of chaos. Read this plot together with the scalar loss, rather than judging performance from geometric closeness to the uncontrolled path.

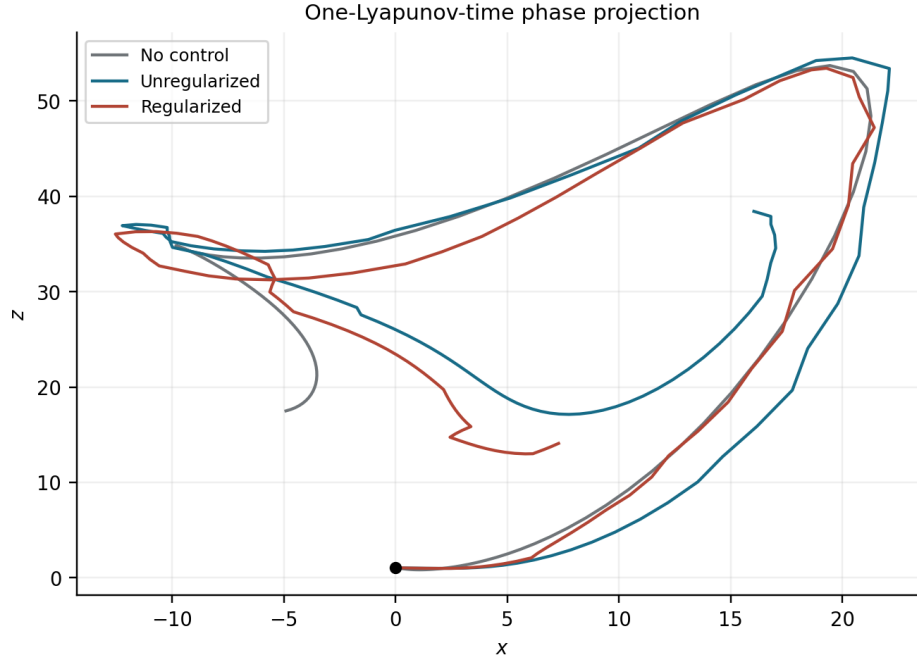


Figure 3: x - z projection. The black point is the shared initial state.

Phase projection (Figure 3). This shows how feedback redirects the coupled transient in state space. It does not by itself prove stabilization, periodicity, or a modified attractor: one Lyapunov time is too short, and only two of three state coordinates are displayed.

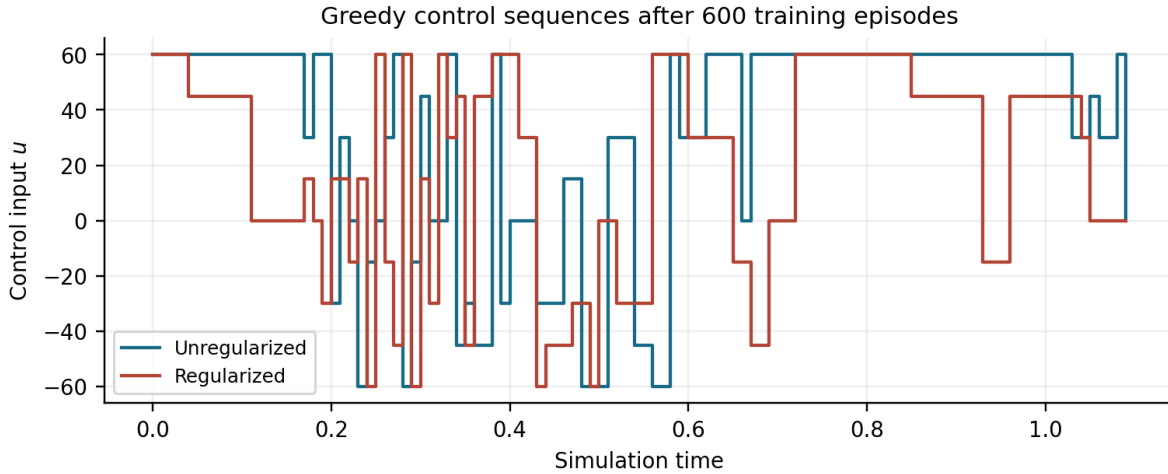


Figure 4: Greedy physical controls; every value lies on the nine-point action grid.

Controls (Figure 4). Large steps and switching are natural consequences of discrete actions and state bins, not numerical interpolation artifacts. The unregularized policy applies $u = 60$ for its first ten steps. The regularized policy uses $u = 60$ for four steps and $u = 45$ for the next six, giving an immediate example of the effort penalty changing behavior. Because $\lambda = 0.007$ is small, the regularized policy still uses large amplitudes; it is effort-aware, not low-control by construction.

7 Limitations and recommended next experiments

1. **Coverage and generalization.** Training from one deterministic state visits few bins. Evaluate on a distribution of initial conditions and report several random seeds with means and uncertainty.
2. **Coarse representation.** Four-unit bins create aliasing; edge clipping aliases every out-of-range state. Compare bin counts and bounds, while monitoring the rapid growth in table size.
3. **Finite-horizon state.** Time remaining is omitted. Adding a time bin would allow a genuinely horizon-dependent policy.
4. **Greedy tie bias.** Randomize ties or initialize values deliberately so unvisited bins do not default to $u = -60$.
5. **Numerical model.** The policy is learned for forward Euler at $\Delta t = 0.01$. Test it with a smaller step or higher-order integrator before making claims about continuous-time behavior.
6. **Safety and persistence.** Use a justified finite divergence threshold and save configurations, Q-tables, checkpoints, and per-seed metrics.
7. **Claims.** Constant learning rate, persistent exploration, finite training, and incomplete visitation do not support a formal convergence claim.

8 Reproduction

The default and effort-regularized experiments are run with

```
python3 sim.py
python3 sim.py --regularized
python3 sim.py --episodes 1200 --eval-episodes 20 --seed 11
```

Relevant command-line options are `-episodes` (600), `-eval-episodes` (5), `-seed` (7), optional `-max-steps`, and `-regularized`. Reproducibility requires preserving the NumPy version, seed, integration parameters, discretization, initial state, and full objective definition.

9 Conclusion

The project demonstrates a clear, reproducible tabular control pipeline for a chaotic continuous system. Its strongest implementation feature is the exact alignment between per-step reward and the trajectory objective. Its main result is a substantial loss reduction for the fixed seed-7 test, with a visible and quantifiable loss-effort tradeoff under regularization. Its principal weakness is limited state-space coverage, so the reported policy should be understood as a successful finite-horizon demonstration rather than a general Lorenz controller.

References

- [1] E. N. Lorenz, “Deterministic Nonperiodic Flow,” *Journal of the Atmospheric Sciences*, vol. 20, no. 2, pp. 130–141, 1963.
- [2] C. J. C. H. Watkins and P. Dayan, “Q-learning,” *Machine Learning*, vol. 8, pp. 279–292, 1992.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.